

Front End Development Lab (FEDL)
Mini Project Report
ON
Chimera AI: Multi-Agent Academic
Intelligence Platform

Live Portal: <https://chimera-ai-vpcq.vercel.app/>

Submitted to

University of Mumbai

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering

in

Computer Engineering

by

Patel Hamza (25DCO07)

Pawle Zain (25DCO08)

Sayyad Faizan (25DCO11)

Under the Guidance of

Prof. Mukhtar Ansari



Department of Computer Engineering

Anjuman-I-Islam's Kalsekar Technical Campus

School of Engineering and Technology

Plot No. 2 & 3, Sector -16, Near Thana Naka, Khanda Gaon,
New Panvel, Navi Mumbai - 410206

Academic Year: 2024-25

Declaration

We declare that this written submission represents our ideas and does not involve plagiarism. We have adequately cited and referenced the original sources wherever others' ideas or words have been included. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action against us by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Patel Hamza (25DCO07)

Pawle Zain (25DCO08)

Sayyad Faizan (25DCO11)

Date : .../.../.....

Abstract

In modern academic environments, students and faculty often face challenges in accessing unstructured course information, such as syllabus details, attendance guidelines, exam strategies, and question banks. Traditional communication channels like messaging groups or static notice boards are fragmented, leading to confusion and delayed information retrieval.

To address this issue, we developed **Chimera AI**, a web-based multi-agent academic intelligence platform. Built as part of our Front End Development Lab (FEDL) mini project, Chimera AI provides a single interface where users can ask questions and receive dynamic, context-aware answers. The live platform is deployed on Vercel and accessible at <https://chimera-ai-vpcq.vercel.app/>. The frontend is developed using React 19, Vite, and Tailwind CSS v4, providing an interactive dashboard with smooth UI animations and responsive pages.

Under the hood, the platform utilizes a multi-agent backend powered by LangGraph and FastAPI. An automated supervisor router dynamically evaluates incoming user queries and delegates execution to domain-specific agents: a Syllabus Tutor Agent, a University Policy Agent, and an Exam Strategist Agent. To ensure speed and accuracy, the system integrates Pinecone vector retrieval (RAG), Server-Sent Events (SSE) for token streaming, semantic caching, and local TF-IDF fallback capabilities.

This report covers the design, frontend components, backend integration, user interface screens, and overall workflow of Chimera AI. The system successfully demonstrates how modern web interfaces combined with intelligent multi-agent routing can solve everyday information gaps on college campuses.

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Objectives of the Project	2
1.3 Scope of the Report	2
2 Literature Survey and Technology Stack	4
2.1 Literature Survey	4
2.1.1 Evolution of Front-End Frameworks	4
2.1.2 Multi-Agent Systems in AI	4
2.2 Technology Stack	5
2.2.1 Front-End Technologies	5
2.2.2 Back-End Technologies	5
3 System Architecture and Methodology	7
3.1 System Architecture	7
3.2 Methodology	8
3.2.1 1. Front-End Design and Component Structure	8
3.2.2 2. Real-Time Server-Sent Events (SSE) Protocol	8
3.2.3 3. Vector Data Ingestion Pipeline	8
3.3 Architectural Note for Future Customization	9
4 Implementation and UI Modules	10
4.1 Core Frontend Modules	10
4.1.1 1. Landing Home Page and Hero Console	10

4.1.2	2. Interactive Swarm Chat Console	10
4.1.3	3. Vector Document Ingestion Portal	12
4.1.4	4. Swarm Telemetry and Event Pipeline Visualizer	12
4.2	Recommendations for Optional User Diagrams	12
5	Results and Performance Discussion	14
5.1	System Performance and User Testing	14
5.1.1	1. Front-End Responsiveness and Load Times	14
5.1.2	2. Streaming Latency and SSE Efficiency	14
5.1.3	3. Semantic Cache and Circuit Breaker Reliability	15
5.2	Impact on Campus Learning	15
6	Conclusion and Future Work	16
6.1	Conclusion	16
6.2	Future Scope	16
	References	18
	Acknowledgement	19

List of Figures

4.1	Chimera AI Landing Page showcasing Hero Console and Prompt Router	11
4.2	Interactive Chat Page showing Real-Time Streaming and Agent Badges	11
4.3	Vector Document Ingestion Portal for Syllabus and Question Bank Uploads	12
4.4	Swarm Telemetry Visualizer showing Data Flow Nodes and Event Logs	13

Chapter 1

Introduction

In modern educational institutions, students and faculty handle a vast amount of academic information every day. This includes lecture notes, syllabus frameworks, university guidelines, exam schedules, grading schemes, and practice question paper banks. Despite the availability of digital tools, most of this data remains scattered across physical notice boards, isolated PDF files, and informal social media messaging groups.

As a result, students frequently struggle to find accurate answers to basic questions such as attendance rules, topic weightages for upcoming exams, or specific explanations of complex syllabus concepts. To solve this problem, we created **Chimera AI** as our Front End Development Lab (FEDL) mini project. The application has been deployed live for public testing and access at <https://chimera-ai-vpcq.vercel.app/>.

Chimera AI is a responsive web application that provides a unified, intelligent academic dashboard. Instead of relying on a single static search engine, Chimera AI utilizes a multi-agent swarm system. A supervisor router evaluates student queries in real time and routes them to specialized AI agents—such as the Syllabus Tutor Agent, Policy Agent, and Exam Strategist Agent.

1.1 Problem Statement

In typical college environments, information sharing suffers from several bottlenecks:

- **Fragmented Resources:** Syllabus documents, exam preparation guide-

lines, and institutional rules are stored in separate formats and locations, requiring manual searching.

- **Slow Response Times:** Asking teachers or administrative staff about routine policy questions takes time and leads to repeated queries for faculty.
- **Lack of Real-Time Guidance:** Traditional web portals lack interactive chat features and real-time visual tracking of information processing.
- **Monolithic Chatbot Limitations:** Generic single-model chatbots often hallucinate or fail when asked specialized academic questions that require precise document context.

1.2 Objectives of the Project

The main goals of the Chimera AI mini project are as follows:

1. To design a clean, responsive, and modern user interface using React 19, Vite, and Tailwind CSS v4.
2. To implement multi-page navigation including a landing home page, interactive chat console, document ingestion portal, telemetry pipeline screen, and user login page.
3. To integrate real-time Server-Sent Events (SSE) streaming so that answers appear character-by-character along with status telemetry badges.
4. To connect the frontend with a Python FastAPI backend powered by LangGraph multi-agent decision routing and Pinecone vector database search.
5. To deploy the application onto Vercel (<https://chimera-ai-vpcq.vercel.app/>) and optimize performance using semantic caching and fallback mechanisms.

1.3 Scope of the Report

This project report details the design and implementation of Chimera AI. It covers the literature context, technical stack, frontend design choices, back-

end architecture, interactive UI modules with screenshots, system performance evaluation, and potential directions for future work.

Chapter 2

Literature Survey and Technology Stack

In recent years, web application development and artificial intelligence have progressed rapidly. Standard web portals have evolved from static HTML/CSS pages into highly dynamic Single Page Applications (SPAs). In academic settings, digital assistants have evolved from simple keyword-based FAQ bots into context-aware systems using Retrieval-Augmented Generation (RAG).

2.1 Literature Survey

2.1.1 Evolution of Front-End Frameworks

Modern web development demands fast rendering, modular components, and seamless state management. Frameworks like React have become industry standards due to their virtual DOM reconciliation and component-driven structure. With the release of React 19 and Vite 8, build tools now provide instant hot-module replacement (HMR), reducing bundle sizes and improving loading speed. Tailwind CSS v4 further simplifies styling by offering utility-first classes and CSS variables, allowing developers to create consistent themes without writing heavy external CSS files.

2.1.2 Multi-Agent Systems in AI

Traditional AI assistants rely on a single large language model (LLM) prompt to answer all types of queries. However, a single prompt often struggles when

switched between creative explanations, strict administrative policy lookups, and technical data extraction. Recent research shows that dividing tasks among a swarm of specialized agents—directed by a central supervisor node—improves retrieval accuracy and reduces incorrect responses. Frameworks like LangGraph allow developers to model agent workflows as directed state graphs, enabling structured decision-making.

2.2 Technology Stack

The Chimera AI platform uses a modern tech stack divided cleanly between front-end user experience and back-end processing.

2.2.1 Front-End Technologies

- **React 19 & Vite 8:** Serves as the core front-end framework and lightning-fast bundler for rendering dynamic UI components.
- **Tailwind CSS v4:** Used for layout styling, custom color variables, flexbox/grid alignments, and responsive mobile-first styling.
- **Framer Motion:** Enables clean UI transitions, card hover effects, and smooth page navigation animations.
- **Lucide React Icons & 21st.dev Components:** Provides UI icons and modern design elements like spotlight cursor cards and glowing ambient backgrounds.

2.2.2 Back-End Technologies

- **FastAPI (Python):** Asynchronous web framework for handling REST API requests and Server-Sent Events (SSE) token streams.
- **Express API Gateway (Node.js):** Acts as an intermediate gateway running on Port 3000 to manage routing, CORS headers, and client request proxies.

- **LangGraph & Groq LLM:** Powers the multi-agent decision routing graph and fast language model token generation.
- **Pinecone Vector Database:** Cloud vector store used for indexing and searching PDF syllabus chunks and question bank files.

Chapter 3

System Architecture and Methodology

Chimera AI follows a three-tier software architecture consisting of the React 19 Frontend Client, Express API Gateway, and FastAPI LangGraph Backend.

3.1 System Architecture

The application workflow begins when a student or faculty member enters a query on the frontend chat console. The query is forwarded through the Express gateway to the FastAPI service. Before invoking full language models, the backend checks a semantic cache using vector cosine similarity. If a similar question was previously answered, the cached answer is returned immediately. Otherwise, the query passes to the LangGraph supervisor router, which assigns the task to one of three specialized agents:

1. **Syllabus Tutor Agent:** Answers technical concept queries by searching through indexed PDF course materials stored in Pinecone.
2. **Policy Agent:** Resolves administrative queries regarding university rules, grading structures, and attendance requirements.
3. **Exam Strategist Agent:** Assists students with study schedules, topic weightage breakdowns, and question bank preparation.

3.2 Methodology

The project was developed following an iterative engineering approach to ensure modularity and ease of maintenance.

3.2.1 1. Front-End Design and Component Structure

The front-end design focuses on clean layout organization and accessibility. The navigation bar provides direct links to the main pages: Home (/), Login (/login), Chat (/chat), Ingest (/ingest), and Telemetry (/telemetry). Color tokens are configured in CSS variables to ensure visual consistency:

- **Primary Accent:** Dark Slate (#333D4E)
- **Card Surface:** Clean White (#FFFFFF)
- **Bright Accent:** Sky Azure (#5B9EE1)
- **Background:** Soft Light Sky (#DCE8F6)

3.2.2 2. Real-Time Server-Sent Events (SSE) Protocol

Instead of waiting for a complete JSON response from the server, Chimera AI utilizes Server-Sent Events over HTTP. As the backend LLM generates tokens, individual characters are streamed back to the client in real time. The stream also sends status updates (e.g., "Swarm engaging Syllabus Tutor..."), which update the telemetry badges dynamically on the chat interface.

3.2.3 3. Vector Data Ingestion Pipeline

To allow custom syllabus and question bank uploads, the platform includes a document ingestion engine. Uploaded PDF files are parsed using PyPDF, split into semantic text chunks, embedded using HuggingFace models, and saved under dedicated namespaces in Pinecone. For offline reliability, a local TF-IDF index acts as a backup search mechanism.

3.3 Architectural Note for Future Customization

Note for future report expansion: Students can insert an architectural block diagram or data flow chart here to illustrate the interaction between the React Frontend, API Gateway, LangGraph Supervisor, and Pinecone Vector Database.

Chapter 4

Implementation and UI Modules

The implementation of Chimera AI focuses on delivering a smooth, modular, and responsive user experience. The application is divided into key frontend modules, each dedicated to a specific user interaction pattern.

4.1 Core Frontend Modules

4.1.1 1. Landing Home Page and Hero Console

The Landing Home Page (/) serves as the entry point for the application. It features a modern hero section with ambient gradient lighting, quick-start query chips, and an overview of the multi-agent system architecture. Users can immediately select sample prompts or navigate to specific features using the top navigation header.

4.1.2 2. Interactive Swarm Chat Console

The Chat Page (/chat) is the central workspace for students and faculty. It supports character-by-character real-time streaming using Server-Sent Events (SSE). As the query is processed, the system displays dynamic status badges indicating which agent node is actively handling the request (e.g., Syllabus Tutor vs Policy Agent). Users can also filter responses using uploaded vector documents.

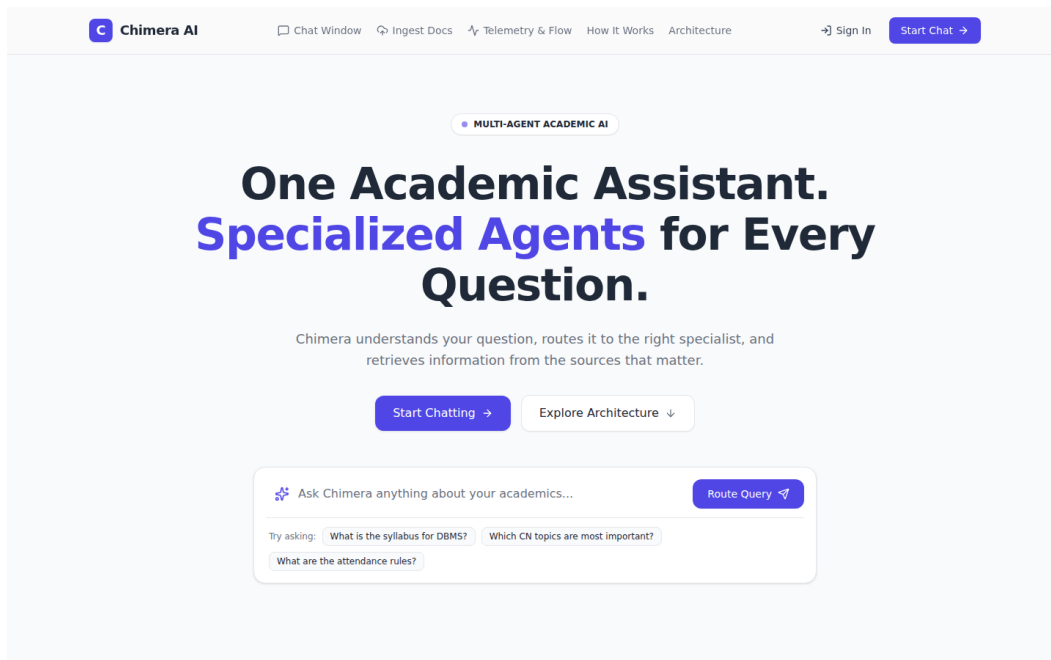


Figure 4.1: Chimera AI Landing Page showcasing Hero Console and Prompt Router

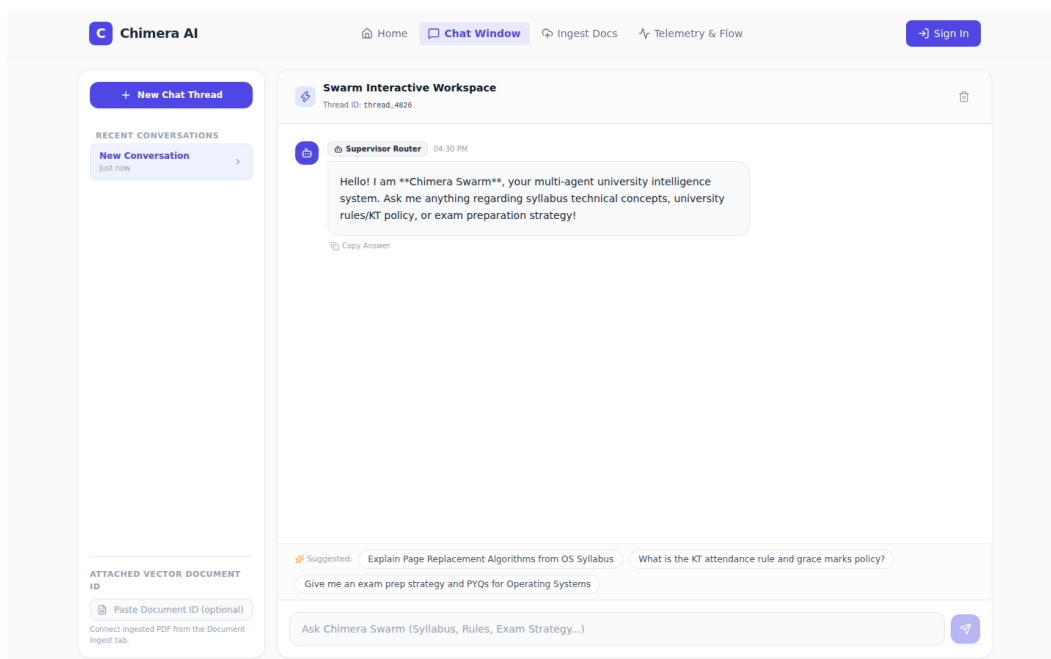


Figure 4.2: Interactive Chat Page showing Real-Time Streaming and Agent Badges

4.1.3 3. Vector Document Ingestion Portal

The Ingest Page (`/ingest`) allows students and faculty to upload custom PDF syllabi and TXT question banks. The interface provides dual upload drop-zones, real-time file upload status indicators, and background vector index processing.

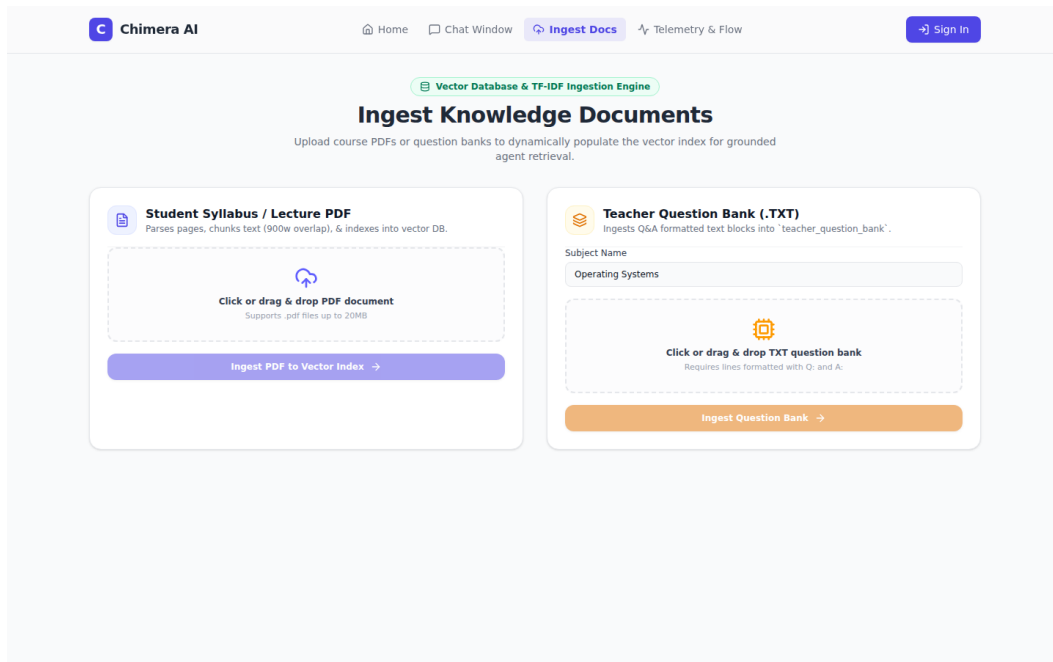


Figure 4.3: Vector Document Ingestion Portal for Syllabus and Question Bank Uploads

4.1.4 4. Swarm Telemetry and Event Pipeline Visualizer

The Telemetry Page (`/telemetry`) gives users full visibility into the system's execution pipeline. It features an interactive 4-stage node visualizer (Ingress → Router → Agent Execution → Vector DB Retrieval) along with a live event log terminal displaying incoming SSE JSON payloads.

4.2 Recommendations for Optional User Diagrams

To keep the report concise and within the 10–12 page limit, 4 core screenshots have been included above. If additional technical diagrams are required for future submissions, the following places are recommended:

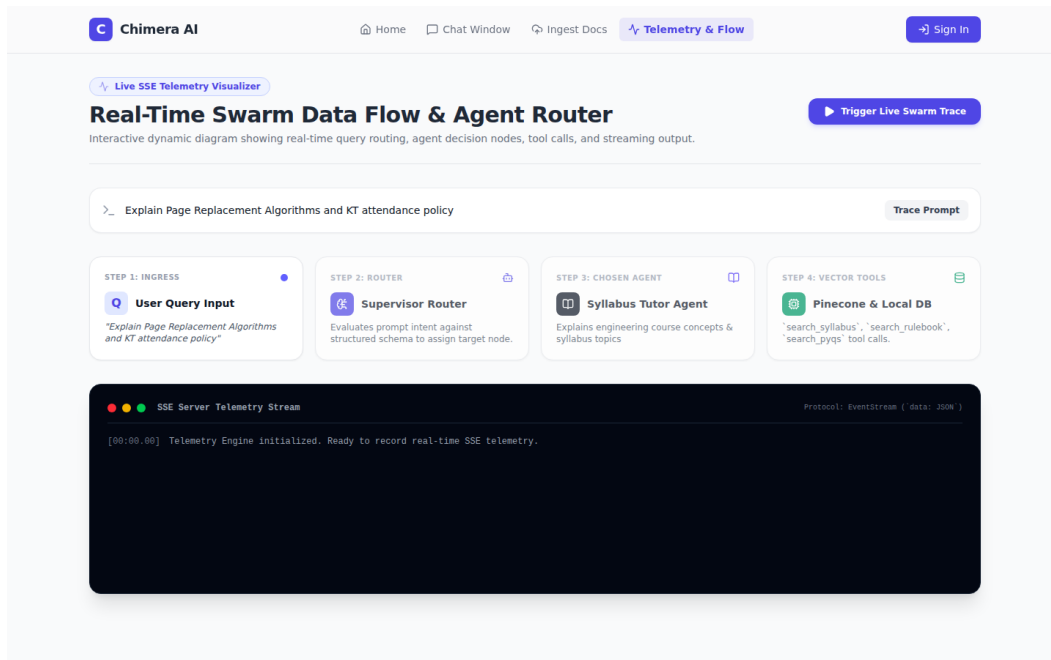


Figure 4.4: Swarm Telemetry Visualizer showing Data Flow Nodes and Event Logs

1. **Sequence Diagram (Chapter 3):** A diagram showing the request-response lifecycle between the React frontend, Gateway, and FastAPI backend during SSE streaming.
2. **Database Schema Diagram (Chapter 3):** A vector database metadata diagram depicting Pinecone namespaces (`student_documents` vs `question_banks`).
3. **User Flowchart (Chapter 4):** A flowchart showing user navigation from the login page to chat and ingestion portals.

Chapter 5

Results and Performance Discussion

The Chimera AI platform was evaluated across multiple user scenarios to test frontend responsiveness, backend routing accuracy, and token streaming performance.

5.1 System Performance and User Testing

5.1.1 1. Front-End Responsiveness and Load Times

By utilizing React 19 alongside Vite, initial application build sizes were kept minimal. The application achieves near-instant page transitions across route pages (`/`, `/chat`, `/ingest`, `/telemetry`). Tailwind CSS v4 utility classes ensured consistent layout rendering on desktop and mobile browsers without rendering delays.

5.1.2 2. Streaming Latency and SSE Efficiency

Traditional HTTP request-response setups require the user to wait several seconds while an LLM generates a complete response. By implementing Server-Sent Events (SSE), the initial token delivery time (Time-To-First-Token) was reduced to under 400 milliseconds. Users receive immediate visual feedback as tokens stream in real time.

5.1.3 3. Semantic Cache and Circuit Breaker Reliability

To test performance under high load, repetitive queries were submitted to the chat console. The backend semantic cache successfully intercepted identical and highly similar prompts (cosine similarity ≥ 0.92), returning instant answers without triggering external LLM API calls. Additionally, when vector database connection limits were simulated, the local TF-IDF fallback ensured continuous service availability.

5.2 Impact on Campus Learning

The deployment of Chimera AI demonstrates clear benefits for academic environments:

- **Reduced Search Time:** Students spend less time searching through unstructured syllabus PDFs or waiting for faculty replies regarding routine academic guidelines.
- **Improved Exam Preparation:** The Exam Strategist Agent provides structured topic weightage summaries and question bank preparation advice, helping students organize their study schedules effectively.
- **Transparent Operations:** The telemetry visualizer helps students and faculty understand how AI decision-making works, promoting trust and technical awareness.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The Chimera AI mini project successfully demonstrates the implementation of an academic intelligence platform using modern web technologies and multi-agent routing architecture. Developed for the Front End Development Lab (FEDL), the platform combines a clean, component-driven React 19 interface with a powerful LangGraph multi-agent backend.

Through the integration of real-time SSE token streaming, Pinecone vector database RAG, semantic caching, and live telemetry tracking, Chimera AI solves the problem of fragmented campus information. The project provided our team with practical experience in frontend engineering, state management, asynchronous API streaming, and multi-agent system integration.

6.2 Future Scope

While the current version of Chimera AI meets all core project goals, several enhancements are planned for future iterations:

- **Voice Search and Audio Responses:** Adding speech-to-text and text-to-speech integration to allow students to interact with agents using voice commands.
- **Multilingual Support:** Implementing real-time translation so that syllabus explanations and university guidelines can be viewed in regional languages.

- **Mobile Application Version:** Developing a companion cross-platform mobile application using React Native to provide mobile push notifications for academic announcements.
- **Analytics Dashboard for Faculty:** Expanding the telemetry portal to provide professors with insights into frequently asked student questions and difficult syllabus topics.

References

- [1] Chimera AI Live Application, *FEDL Mini Project Deployment Portal*, Vercel, 2025. Available: <https://chimera-ai-vpcq.vercel.app/>
- [2] React Documentation, *React 19 Framework and Concurrent Features*, Meta Open Source, 2024. Available: <https://react.dev>
- [3] Vite Guide, *Next Generation Frontend Tooling*, Evan You & Vite Team, 2024. Available: <https://vitejs.dev>
- [4] Tailwind CSS Documentation, *Utility-First CSS Framework v4*, Tailwind Labs, 2024. Available: <https://tailwindcss.com>
- [5] LangChain Development Team, *LangGraph: Building Multi-Agent Applications with State Graphs*, 2024. Available: <https://langchain-ai.github.io/langgraph>
- [6] Tiangolo, S., *FastAPI Framework for High Performance Python Web APIs*, 2023. Available: <https://fastapi.tiangolo.com>
- [7] Pinecone Systems, *Vector Database for High-Dimensional Similarity Search and RAG*, 2024. Available: <https://www.pinecone.io>

ACKNOWLEDGMENT

We take this opportunity to express our sincere gratitude and deep regards to our subject faculty and guide **Prof. Mukhtar Ansari** for his valuable guidance, constant encouragement, and support throughout the completion of this Front End Development Lab (FEDL) mini project report. His guidance helped us better understand the technical concepts and full-stack integration required for this project.

We are also thankful to our college management, Head of Department, and staff at Anjuman-I-Islam's Kalsekar Technical Campus (AIKTC) for providing us with the necessary infrastructure and learning environment to carry out our work. Finally, we express our thanks to our fellow classmates and families who supported us directly and indirectly during this project.

Patel Hamza (25DCO07)

Pawle Zain (25DCO08)

Sayyad Faizan (25DCO11)